Leveraging AI for Coding Responsibly

NTRES 6100 - Collaborative and Reproducible Data Science in R

October 8, 2026

Azwad Iqbal

PhD Candidate

Therkildsen Lab



Cornell CALS

College of Agriculture and Life Sciences

Spot the **bug**!

This AI-written code runs without error

Prompt: “Calculate mean life expectancy by continent for 2002 and 2007"

```
library(tidyverse)
library(gapminder)
gapminder |>
  filter(year == 2002 | 2007) |>
  group_by(continent) |>
  summarize(mean_lifeExp = mean(lifeExp))
```

Take a minute

It returns five tidy rows with plausible numbers. What is wrong, and how would you have caught it?

Clean-looking, error-free code is **not** the same as correct code.

Lecture overview

How LLMs work

Next-token prediction, training, context, and the model landscape



Prompting

Giving the model what it cannot see; a hands-on prompt lab



Using AI responsibly

Environment, integrity, privacy, learning, and when to use AI



AI in RStudio

Posit Assistant, plus a live demo



Lecture overview

How LLMs work

Next-token prediction, training, context, and the model landscape



Prompting

Giving the model what it cannot see; a hands-on prompt lab

Using AI responsibly

Environment, integrity, privacy, learning, and when to use AI

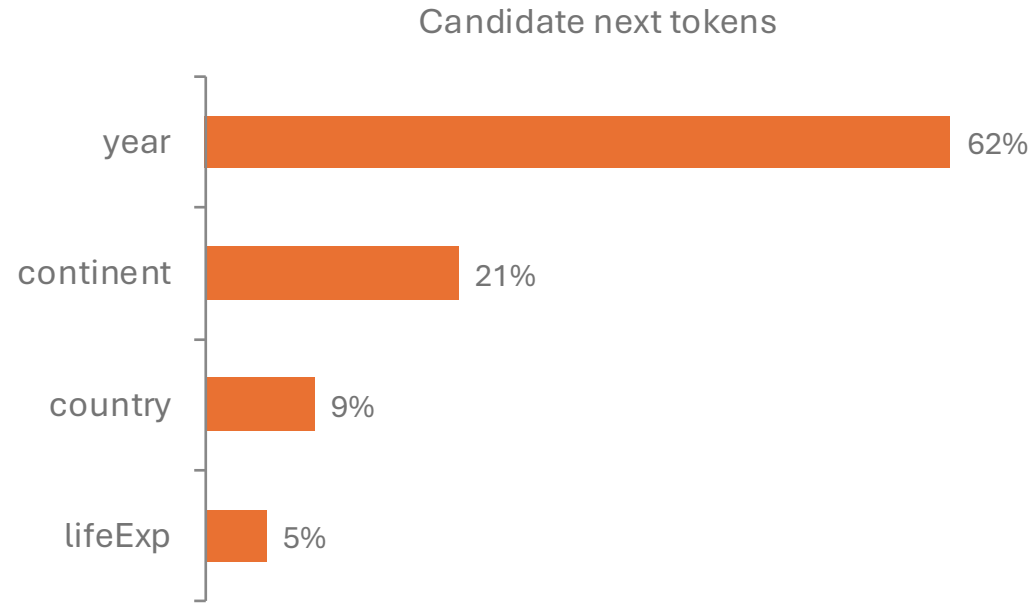
AI in RStudio

Posit Assistant, plus a live demo

An LLM predicts the next token, one at a time

- Text is split into **tokens**: chunks of about 3-4 characters.
- The model scores every possible next token, picks one, appends it, and repeats.
- No fact lookup, and no running of your code, unless a tool does it.
- **It is optimized to *sound* right, not to *be* right.**

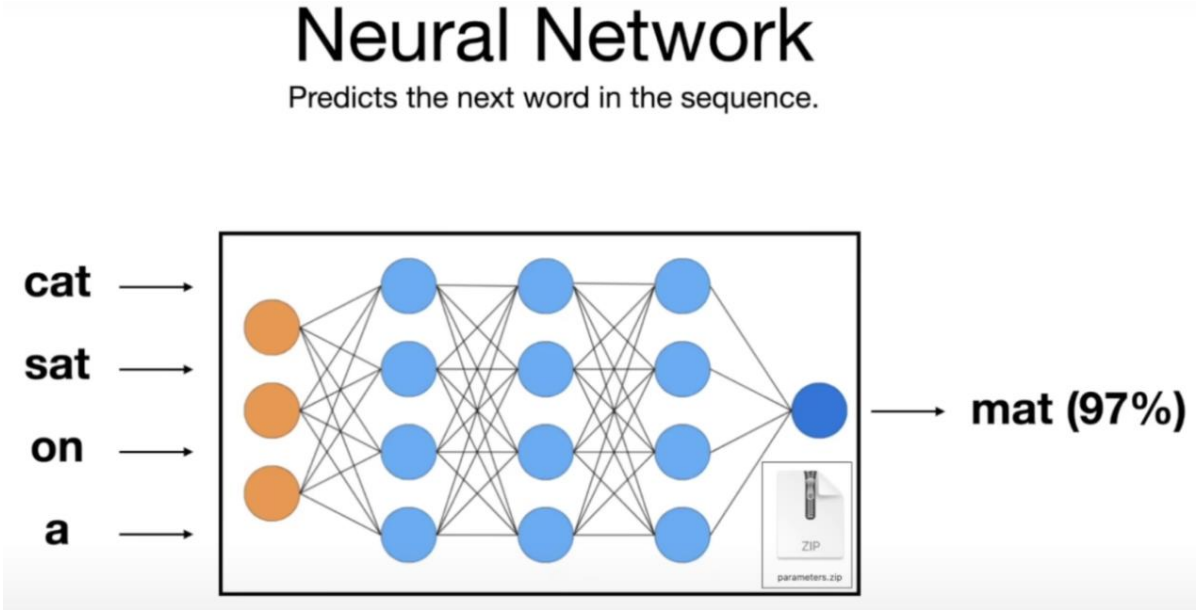
```
gapminder |> filter( ___
```



Illustrative probabilities, not from a real model

An LLM predicts the next token, one at a time

- Text is split into **tokens**: chunks of about 3-4 characters.
- The model scores every possible next token, picks one, appends it, and repeats.
- No fact lookup, and no running of your code, unless a tool does it.
- **It is optimized to *sound* right, not to *be* right.**



Behavior is learned in stages

1

Pretraining

Reads huge amounts of text and code; learns language, syntax, and common patterns

2

Instruction tuning

Trained on curated examples of requests and good answers; learns to follow instructions

3

Preference training

People (and models) rate answers; learns to be helpful and to decline harmful requests

Training data stops at a cutoff date, so newer package versions and functions may be unknown or confused.

LLMs only know what’s within its context window

What the model *can* see

- Patterns from its training data (frozen at a cutoff)
- Your prompt and the conversation so far
- Code, errors, and data you paste in
- Results from tools it is allowed to use (e.g. web searches, connections to Google Drive, context databases you provide)

What the model *can't* see
(unless you share it)

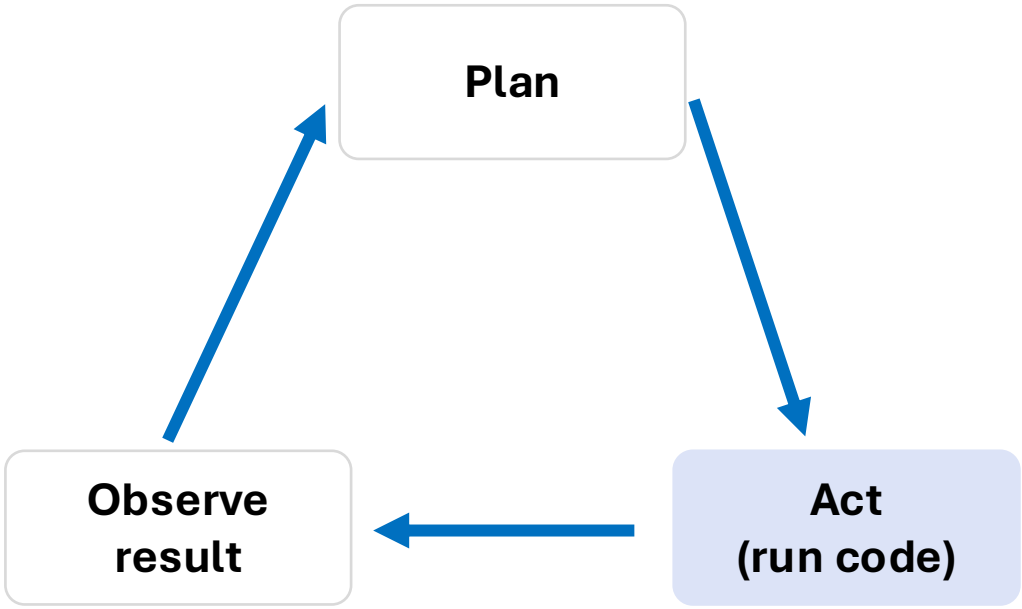
- Your data frame's columns and types
- Your file paths and project structure
- The exact error message you got
- Which package versions you have installed

Chatbots vs Agents

Chat app

- You copy code out, run it, and paste errors back
- Model never sees your data or the result
- Examples: ChatGPT, Claude, or Gemini in a browser

Agent: a model running tools in a loop



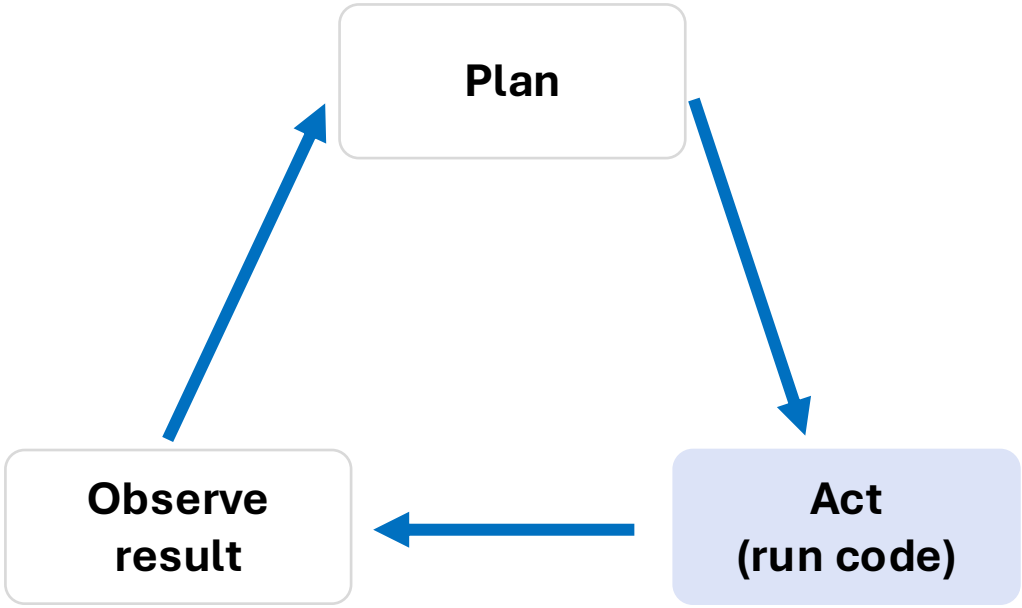
Agents actively work ***within*** your coding/working environment.
This can make them powerful, but potentially risky

Chatbots vs Agents

Chat app

- You copy code out, run it, and paste errors back
- Model never sees your data or the result
- Examples: ChatGPT, Claude, or Gemini in a browser

Agent: a model running tools in a loop



Agents actively work ***within*** your coding/working environment.
This can make them powerful, but potentially risky

Certain tools, like Posit Assistant can do *both*

The interface matters more than the model

ChatGPT	OpenAI		Widely used; can run Python, but not your R session
Claude	Anthropic		Powers Claude Code and Posit Assistant
Gemini	Google		Built into Google Workspace, search engines
Microsoft Copilot	Microsoft		Integrated into Windows and Microsoft Office products
Open-weight models	Llama, DeepSeek		Run locally (e.g., Ollama); data stays on your computer, but generally less powerful

Specialized tools can add context and guardrails

Coding and data-science agents

- Posit Assistant (R/Python data work)
- Claude Code
- GitHub Copilot
- OpenAI Codex

Literature and research

- Elicit, Consensus
- Semantic Scholar features
- Scholarly connectors in general assistants

Genomic/proteomic language models in biology

- Protein language models (e.g., ESM)
- DNA/genome models (e.g., Evo)
- Same next-token idea, different vocabulary (nucleotides, amino acids)

Lecture overview

How LLMs work

Next-token prediction, training, context, and the model landscape

Prompting

Giving the model what it cannot see; a hands-on prompt lab



Using AI responsibly

Environment, integrity, privacy, learning, and when to use AI

AI in RStudio

Posit Assistant, plus a live demo

A good prompt supplies what the model can't see

1 Context

Language, packages, glimpse() output, what you tried

2 Task

The specific result you want

3 Constraints

"tidyverse only," "native pipe," "keep my column names"

4 Output format

Code with comments, an explanation, or alternatives

Same goal, very different prompts

Vague

```
Make a plot of life expectancy
from gapminder.
```

The LLM has to guess the years, countries, summary, and geom.

Structured

```
I'm using R with dplyr and ggplot2.
`gapminder` has columns country,
continent, year, lifeExp, pop,
gdpPercap.

Goal: median lifeExp by continent
over time, one line per continent.
Constraints: native pipe |>.
Return: commented code, then a
list of assumptions you made.
```

Habits that make prompting work

1	**Paste the full error**	Along with the code that produced it; "it doesn't work" isn’t very helpful
2	**Give a reprex** (reproducible example)	Put head(df) outputs or the `reprex` package rather than verbally describing your data
3	**Ask for the reasoning**	Check that the explanation matches what the code actually does
4	**Iterate**	Treat the first answer as a draft and ask what it assumed
5	**Start fresh**	Open a new chat when a long thread drifts

Prompt practice: 12 minutes in pairs with gapminder

Round 1 · Vague

3 min

"Make a plot of life expectancy from gapminder."

What did it assume?

Round 2 · Structured

5 min

Median life expectancy by continent over time, using the four-part template in #lecture-chat.

Does it run? Median or mean? Grouped by both?

Round 3 · Verify

4 min

"What assumptions does this code make? Write two checks I can run."

Did the checks catch anything?

Any chat assistant is fine. No account? Pair up, or critique a neighbor's output.

Lecture overview

How LLMs work

Next-token prediction, training, context, and the model landscape

Prompting

Giving the model what it cannot see; a hands-on prompt lab

Using AI responsibly

Environmental impacts, integrity, privacy, learning, and when to use AI



AI in RStudio

Posit Assistant with a free Gemini key, plus a live demo

Environmental impacts of a single prompt:

0.24 Wh

energy per median text prompt

0.26 mL

water, about five drops

~9 s

of watched TV, Google's comparison

Source: Elsworth et al. 2025 from Google Gemini data

- Earlier independent estimates ranged from about 1.8 to 7 Wh per prompt
- Reasoning models, images, and agent sessions with many calls cost far more.
- Habits: smaller models for simple tasks, fewer regenerations, check documentation first before consulting AI

Environmental impacts of a single prompt:

0.24 Wh

energy per median text prompt

0.26 mL

water, about five drops

~9 s

of watched TV, Google's comparison

Source: Elsworth et al. 2025 from Google Gemini data

- Earlier independent estimates ranged from about 1.8 to 7 Wh per prompt
- Reasoning models, images, and agent sessions with many calls cost far more.
- Habits: smaller models for simple tasks, fewer regenerations, check documentation first before consulting AI

You are responsible for every line you submit

Follow the policy	Each assignment states when and how AI may be used
Disclose	Note the tool, what it helped with, and what you verified in your Quarto file
Mind licenses/copyright	AI can reproduce licensed code. Check before reusing large blocks in published work
In research	Journals increasingly require disclosure and AI cannot be an author

Anything you paste leaves your machine

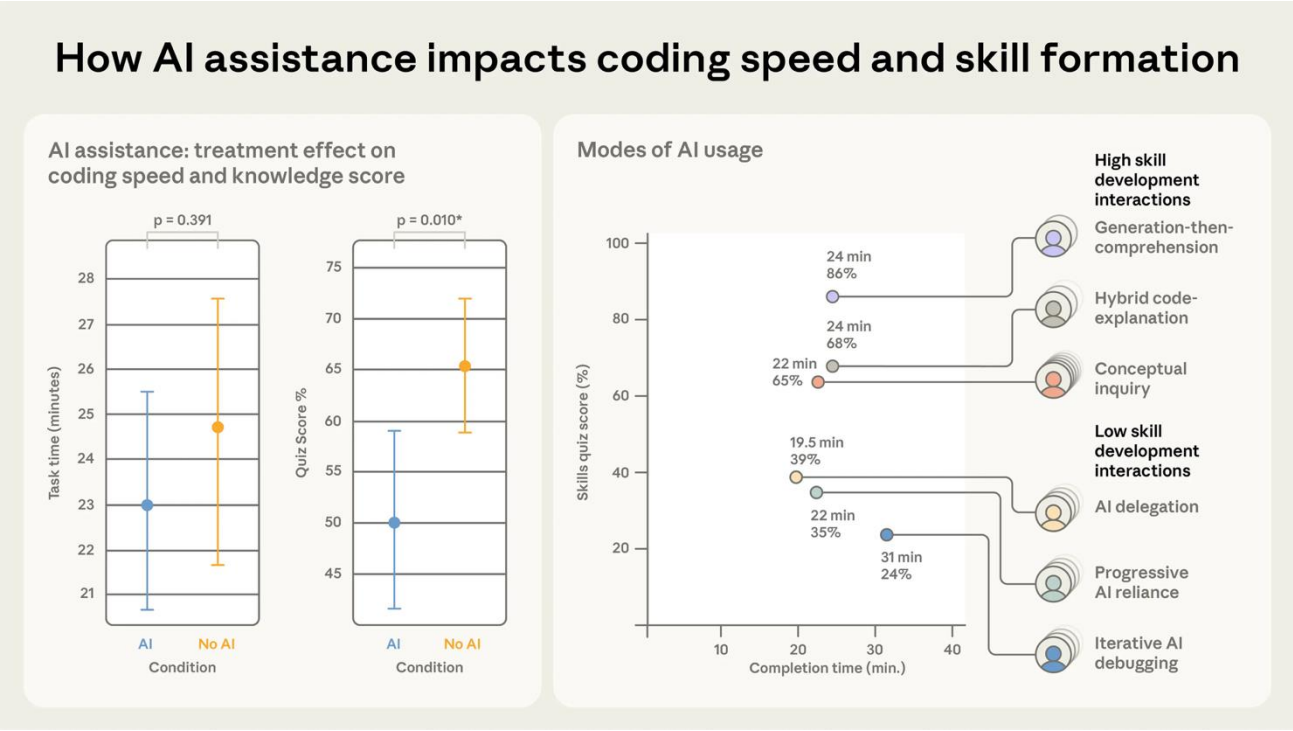
Never paste/share:

- Human-subjects or other protected data
- Unpublished collaborator data
- Passwords, tokens, API keys
- Anything your data agreement forbids sharing

When working with agents:

- Read your whole project folder
- Know what is in it before you start a session
- Check whether chats are used for training
- Check if institutional licenses allow agents

Leaning on AI while learning can cost skills



Study on the affects of AI use on learned coding skills

- Randomized trial, 52 developers (Anthropic, Jan 2026)
- Largest gap was on debugging questions.
- AI users were not significantly faster.
- Users who asked conceptual "why" questions scored as well as non-users.
- *Caveat: one small study, run by an AI company.*

Discuss: which of these costs would change how you use AI this semester?

Use AI freely only where you can **check it**

	Hard for me to verify	Easy for me to verify
Skill I am learning	**Learn it yourself first** Docs, textbooks/readings, office hours AI only to explain concepts (if at all)	**Use AI as a tutor** Attempt first, then ask why and compare with your version
Skill I already have	**Slow down** Get a human expert to review; AI output is hard to trust here	**Delegate to AI** Boilerplate, regex, plot themes; verify, then move on

Combine: you think, AI checks and explains

1	**Attempt first**	10-15 minutes on your own, then compare with AI's version
2	**Hints, not answers**	"Which dplyr verb do I need, and why?"
3	**AI as reviewer**	"What edge cases could break my code?"
4	**AI for tests**	Row counts, stopifnot(), expected group sizes
5	**Explain it back**	If you can't explain a line, you are not done

Use AI freely only where you can **check it**

	Hard for me to verify	Easy for me to verify
Skill I am learning	**Learn it yourself first** Docs, textbooks/readings, office hours AI only to explain concepts (if at all)	**Use AI as a tutor** Attempt first, then ask why and compare with your version
Skill I already have	**Slow down** Get a human expert to review; AI output is hard to trust here	**Delegate to AI** Boilerplate, regex, plot themes; verify, then move on

Think-pair-share
Place two recent coding tasks on the grid. Compare with a neighbor.

Lecture overview

How LLMs work

Next-token prediction, training, context, and the model landscape

Prompting

Giving the model what it cannot see; a hands-on prompt lab

Using AI responsibly

Environment, integrity, privacy, learning, and when to use AI

AI in RStudio

Posit Assistant, plus a live demo



Four ways to get AI help in RStudio

Tool:	Where it runs:	Sees your R session?	Best for:
Posit Assistant	Sidebar pane, RStudio 2026.04+	Yes, built in	Exploring, plotting, cleaning data
GitHub Copilot	Inline, Source mode only	No, open files only	Autocomplete while typing
Claude Code	RStudio Terminal tab	With the btw MCP server	Multi-file edits, scripts, git
Chat app	Browser window	No, copy and paste	Quick questions, explanations

The more an assistant can see and do, the more carefully you need to audit what it did.

Posit Assistant is a built-in agent within RStudio

Sees your session

Reads loaded objects and runs code in your console

Plots in the chat

You and the model look at the same figure

Data-cleaning mode

Surfaces questions to you instead of silently deciding

Pauses often

Runs a few steps, summarizes, suggests next moves

Free option: a Google AI Studio Gemini key (e.g., Gemini 3.5 Flash Lite)

Also available: Claude Code (paid, not demoed)

- Runs in RStudio's Terminal tab, inside your project folder
- Reads and edits multiple files; runs scripts with Rscript
- Works well with git: commit first, then review the diff
- Out of the box it cannot see objects in your R session
- The btw package bridges that gap via an MCP server

Project conventions: CLAUDE.md

```
# Conventions
- tidyverse style, native pipe |>
- never modify files in data/raw/
- write scripts to R/
- explain each change you make
```

Setup: Posit Assistant with a free Gemini key

1. Update RStudio to 2026.04.0 or later
2. Get a free API key at aistudio.google.com (Get API key)
3. Click Posit Assistant on the toolbar, then Install Posit Assistant
4. Configure another provider > Google Gemini, and paste your key
5. Pick a Flash-Lite model and try a small task first

Free-tier requests may be used to improve Google's products: never paste sensitive data.

Backup option: GitHub Copilot Free in RStudio

1. Tools > Global Options > Assistant, then select GitHub Copilot under “Use code assistant”
2. Sign in with the GitHub account you use for this course
3. Switch your Quarto file to Source mode (Visual mode doesn’t work)
4. In an R chunk, type a comment such as # plot lifeExp by continent, then a new line
5. Press Tab to accept the gray suggestion, then read it before running

Copilot Free: about 2,000 completions and 50 chats a month. Autocomplete only; it **cannot** see your R session.

Live demo: follow along in Posit Assistant

1	2	3	4	5
Inspect	**Tidy**	**Plot**	**Script**	**Commit**
Load the penguins_raw data: what needs cleaning?	Fix column names and other inconsistencies in the raw dataset	ggplot of the cleaned data	Save steps to R/clean_penguins.R with a row check	Message says what AI did
Audit	**Audit**	**Audit**	**Audit**	**Audit**
Compare with glimpse()	Row and column counts before and after	Does it answer the question?	Review the code, does it make sense?	Begin a habit of disclosing where AI was used and what it did

Follow along: run each step in your own Posit Assistant with your free Gemini key

Three things to **take with you**

1

Verify everything

LLMs predict plausible text. Clean code can still be wrong.

2

Give it context

Data structure, errors, constraints, and the output you want.

3

Attempt first

When learning, do the thinking yourself. Delegate only what you can check.